

Lab Manual 6: Practice Data Visualization with Matplotlib & Seaborn

Introduction

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools make it easier to see and understand trends and patterns in data. Matplotlib is one of the most popular Python libraries for data visualization. It provides a flexible and powerful interface for creating a wide range of static, animated, and interactive plots.

Why Use Matplotlib?

- To understand data better by visualizing it.
- To identify trends, patterns.
- To represent data clearly in reports and presentations.

Importing Required Library

```
import matplotlib.pyplot as plt
```

1. Line Plot

Scenario-Based Question:

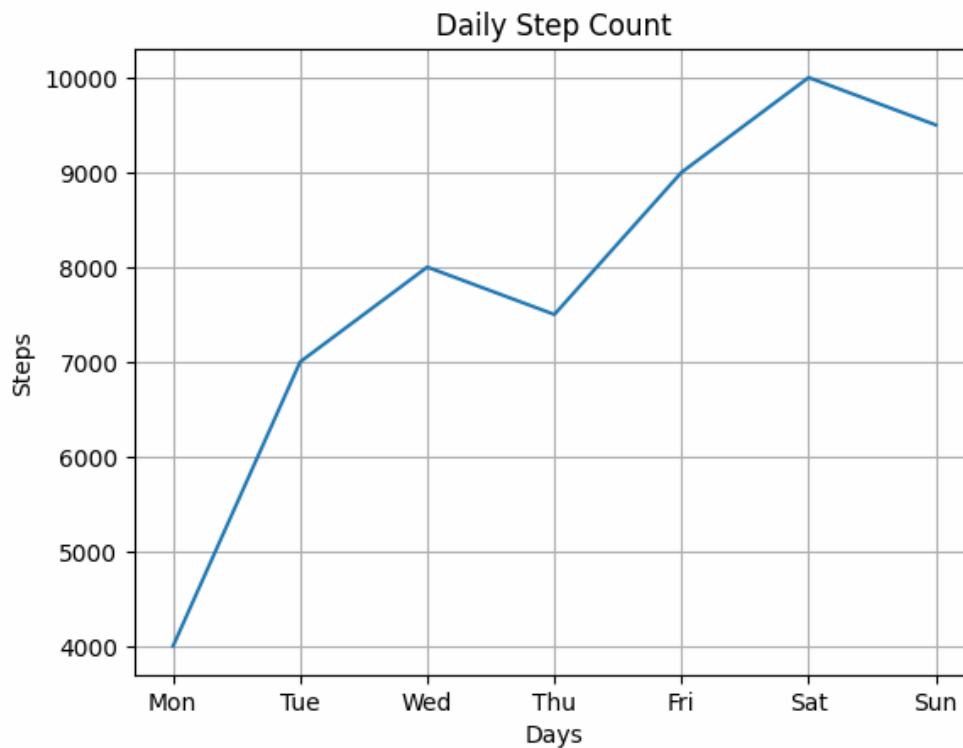
You are tracking your daily step count over a week and want to visualize your progress to stay motivated.

Code:

```
import matplotlib.pyplot as plt
x = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
y = [4000, 7000, 8000, 7500, 9000, 10000, 9500]

plt.plot(x, y) # Basic line plot
plt.title("Daily Step Count")
plt.xlabel("Days")
plt.ylabel("Steps")
plt.grid(True) # Adds grid lines to improve readability
plt.show()
```

Output:



Customization Options:

```
plt.plot(x, y, color='green', linestyle='--', linewidth=2, marker='o', markersize=8)
```

color='green': Changes line color.

linestyle='--': Dashed line instead of solid.

linewidth=2: Makes the line thicker.

marker='o': Adds circle markers at data points.

markersize=8: Increases the marker size.

2. Bar Chart

Scenario-Based Question:

You are managing a small store and want to compare the number of items sold each day over a week.

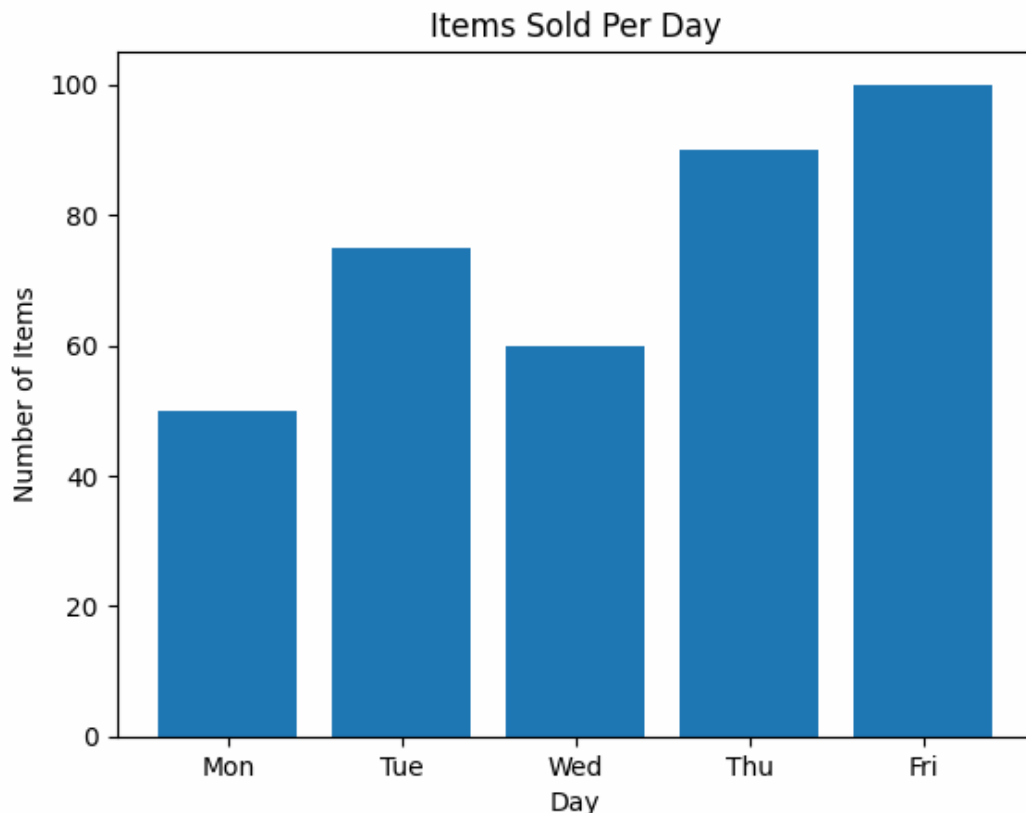
Example Code:

```
import matplotlib.pyplot as plt
x = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
y = [50, 75, 60, 90, 100]

plt.bar(x, y) # Basic bar chart
```

```
plt.title("Items Sold Per Day")
plt.xlabel("Day")
plt.ylabel("Number of Items")
plt.grid(False)
plt.show()
```

Output:



Customization Options:

```
plt.bar(x, y, color='skyblue', edgecolor='black', linewidth=1.5, width=0.5)
```

color='skyblue': Sets the fill color.

edgecolor='black': Adds black border to bars.

linewidth=1.5: Sets thickness of bar borders.

width=0.5: Controls bar width (default is 0.8).

3. Histogram

Scenario-Based Question:

You are analyzing test scores of your students to see how their marks are distributed.

Example Code:

```
import matplotlib.pyplot as plt
scores = [45, 56, 78, 45, 60, 80, 90, 100, 70, 66, 77, 88, 95, 45, 60]

plt.hist(scores, bins=8)
plt.title("Score Distribution")
plt.xlabel("Score Ranges")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

Customization Options:

```
plt.hist(scores, bins=5, color='orange', edgecolor='black', alpha=0.7,
linewidth=1.5)
```

color='orange': Sets bar color.
bins=5: Number of intervals or groups.
edgecolor='black': Adds border to bins.
alpha=0.7: Sets transparency.
linewidth=1.5: Border thickness.

4. Scatter Plot

Scenario-Based Question:

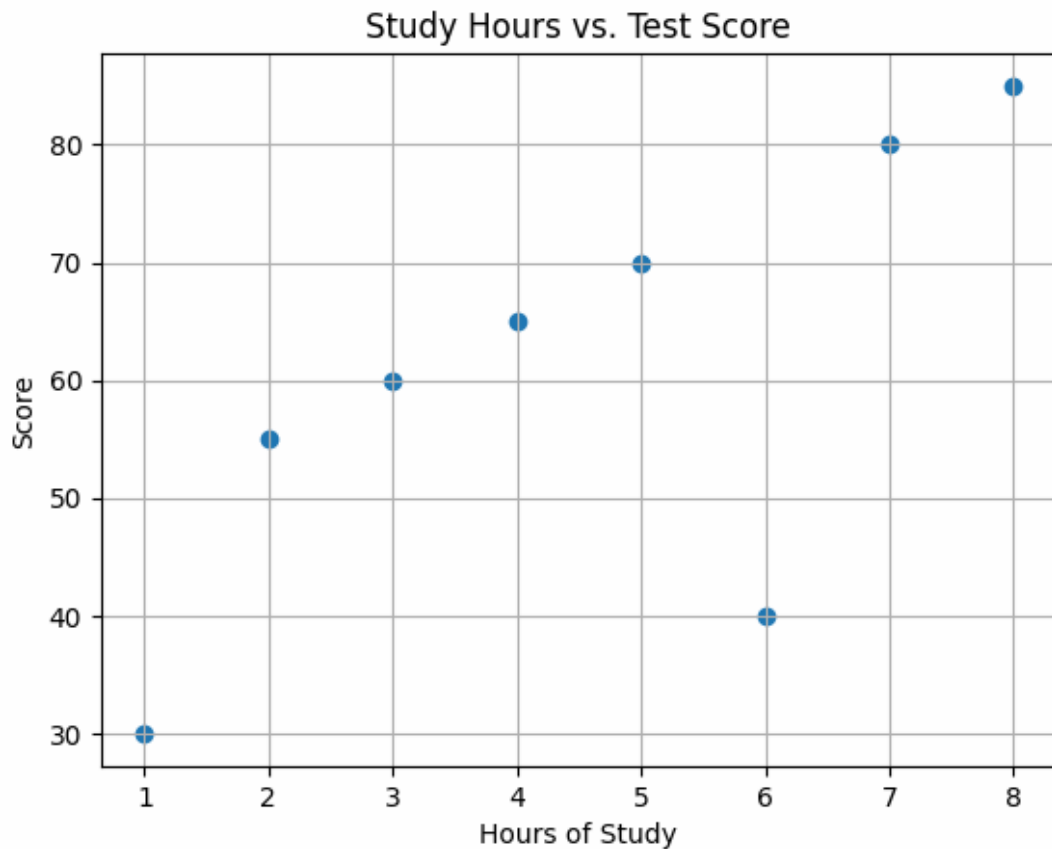
You are conducting a study to find the relationship between number of study hours and test scores.

Example Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5, 6, 7, 8]
y = [30, 55, 60, 65, 70, 40, 80, 85]

plt.scatter(x, y) # Basic scatter plot
plt.title("Study Hours vs. Test Score")
plt.xlabel("Hours of Study")
plt.ylabel("Score")
plt.grid(True)
plt.show()
```

Output:



Customization Options:

```
plt.scatter(x, y, color='purple', s=100, alpha=0.6, marker='^')
```

color='purple': Dot color.

s=100: Dot size.

alpha=0.6: Transparency.

marker='^': Triangle marker instead of default dot.

5. Pie Chart

What is a Pie Chart?

A **pie chart** is used to display data in the form of slices, where each slice represents a category's proportion in the whole dataset.

Scenario-Based Question:

You are given the market share data of five companies in the tech industry. You need to visualize the percentage of the market each company controls.

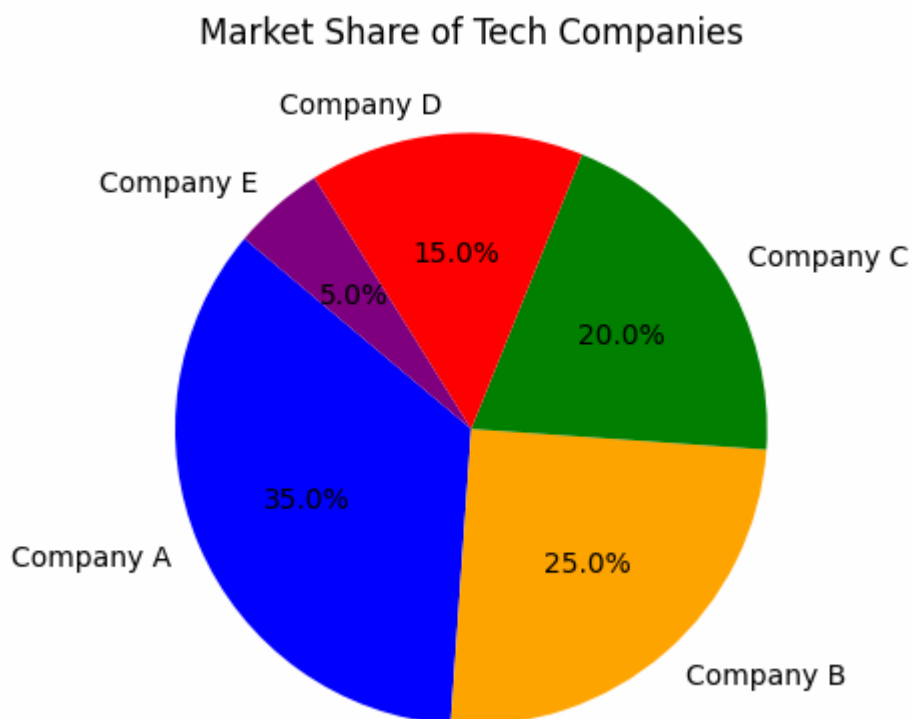
How to Create a Pie Chart?

```
import matplotlib.pyplot as plt

# Data
companies = ['Company A', 'Company B', 'Company C', 'Company D', 'Company E']
market_share = [35, 25, 20, 15, 5]

# Create Pie Chart
plt.pie(market_share, labels=companies, autopct='%1.1f%%', startangle=140,
        colors=['blue', 'orange', 'green', 'red', 'purple'])
plt.title('Market Share of Tech Companies')
plt.show()
```

Output:



Customizing the Pie Chart

`autopct='%1.1f%%'` → Adds percentage labels on the slices.

`startangle=140` → Rotates the chart to start from a specific angle.

`colors=['blue', 'orange', 'green', 'red', 'purple']` → Defines specific colors for each slice.

6. Area Plot

What is an Area Plot?

An **area plot** is a variation of the line plot, where the area below the line is filled with color to highlight the magnitude of the values.

Scenario-Based Question:

You are tasked with visualizing the cumulative total sales of a product over time. An area plot can show how sales have grown over the months, with the area below the line filled in to emphasize the cumulative nature of the data.

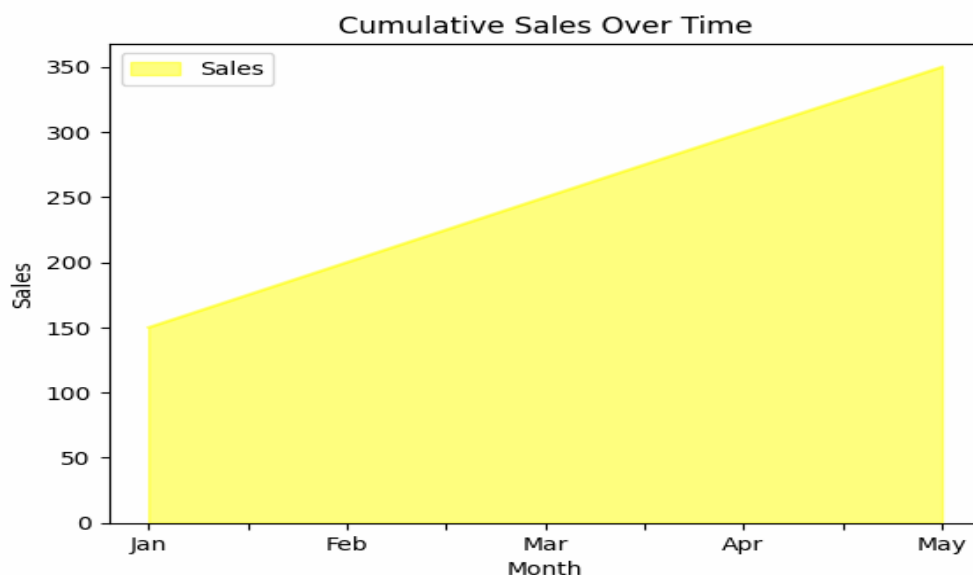
How to Create an Area Plot Using Matplotlib and Pandas?

```
import matplotlib.pyplot as plt
import pandas as pd

# Data
data = {'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May'],
        'Sales': [150, 200, 250, 300, 350]}
df = pd.DataFrame(data)

# Create Area Plot
df.plot(x='Month', y='Sales', kind='area', color='yellow', alpha=0.5)
plt.title('Cumulative Sales Over Time')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.grid(False)
plt.show()
```

Output:



Customizing the Area Plot

color='skyblue' → Sets the color for the area below the line.
alpha=0.5 → Makes the area semi-transparent for better readability.
grid=True → Adds grid lines to the plot.

6. 3D Plot

What is a 3D Plot?

A **3D plot** allows you to visualize data in three dimensions, making it easier to observe relationships between three variables.

Scenario-Based Question:

You are analyzing a dataset with three variables: Age, Height, and Weight. A 3D scatter plot will help you visualize how these three variables relate to each other.

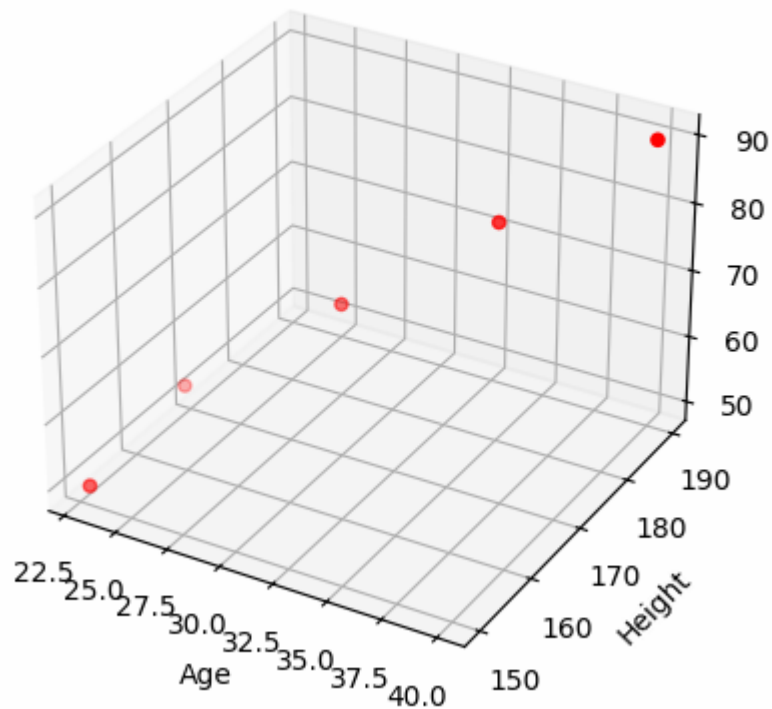
How to Create a 3D Plot ?

```
import matplotlib.pyplot as plt #
from mpl_toolkits.mplot3d import Axes3D # This is for 3D plotting in
Matplotlib
import pandas as pd

# Step 1: Create some sample data
data = {
    'Age': [23, 25, 30, 35, 40],
    'Height': [150, 160, 170, 180, 190],
    'Weight': [50, 60, 70, 80, 90]
}
# Step 2: Convert the dictionary to a Pandas DataFrame
df = pd.DataFrame(data)
# Step 3: Create a figure for the plot
fig = plt.figure() # This creates a new figure window for the plot
# Step 4: Add a 3D subplot to the figure
# The '111' means that we are using 1 row, 1 column, and the first subplot
(which is the only one here)
ax = fig.add_subplot(111, projection='3d') # The 'projection="3d"' makes it a
3D plot
# Step 5: Plot the data using a 3D scatter plot
# The 'scatter' method is used for plotting individual data points in 3D space.
ax.scatter(df['Age'], df['Height'], df['Weight'], color='red')
# Step 6: Customize the axes with labels
ax.set_xlabel('Age')
ax.set_ylabel('Height')
ax.set_zlabel('Weight')
# Step 7: Add a title to the plot
plt.title('3D Scatter Plot')
# Step 8: Display the plot
plt.show()
```

Output:

3D Scatter Plot



Customizing the 3D Plot

`ax.set_xlabel('Age')` → Sets the label for the x-axis.
`ax.set_ylabel('Height')` → Sets the label for the y-axis.
`ax.set_zlabel('Weight')` → Sets the label for the z-axis.

Here's a list of common 3D plots supported using Matplotlib + pandas, without Seaborn:

1. 3D Scatter Plot

Code:

```
ax.scatter(x, y, z)
```

2. 3D Line Plot

Useful for showing trends or sequences in 3D space.

Code:

```
ax.plot(x, y, z)
```

3. 3D Surface Plot

Used when you want to show a 3D surface like a mountain range or heat-like surface.

Code:

```
ax.plot_surface(X, Y, Z)
```

4. 3D Wireframe Plot

Looks like a mesh net; useful for understanding shape without color shading.

Code:

```
ax.plot_wireframe(X, Y, Z)
```

5. 3D Bar Plot

Bars rising from the 3D base great for showing comparisons in three dimensions.

Code:

```
ax.bar3d(x, y, z, dx, dy, dz)
```

6. 3D Contour Plot

Used to draw contour (level) lines on a 3D surface.

Code:

```
ax.contour3D(X, Y, Z, levels=50)
```

7. 3D Quiver Plot

Used to show vector fields in 3D (e.g., wind directions or forces).

Code:

```
ax.quiver(x, y, z, u, v, w)
```

Introduction to Seaborn

Seaborn is a Python data visualization library built on top of matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

Advantages:

- Built-in themes for better aesthetics
- Simplified syntax
- Works well with pandas DataFrames

Types of Visualizations in Seaborn

Seaborn builds on top of Matplotlib and provides a high-level interface for drawing attractive and informative statistical graphics.

Here are the most commonly used visualizations in Seaborn:

Visualization Type	Description
1. Line Plot	Shows trends over time or sequences.
2. Bar Plot	Shows comparisons among categories.
3. Count Plot	Special bar plot for counting occurrences.
4. Box Plot	Shows distribution with median, quartiles, and outliers.
5. Violin Plot	Similar to boxplot, but with distribution density.
6. Strip Plot	Shows all data points, useful with small datasets.
7. Swarm Plot	Non-overlapping version of strip plot.
8. Histogram	Distribution of numerical data.
9. KDE Plot	Kernel Density Estimation for smooth distribution.
10. Pair Plot	Pairwise relationships across numerical columns.
11. Heatmap	Color-coded matrix representing values.
12. Scatter Plot	Shows relationship between 2 variables.
13. Joint Plot	Combination of scatter plot + distribution.
14. Regression Plot	Scatter plot with a regression line.
15. Catplot	Generalized plot for categorical data (combines bar, box, violin, etc.).

1. Bar Plot

What is it? A bar plot displays the relationship between a categorical variable and a numerical variable using rectangular bars.

Where is it used in daily life? Bar plots are used in business reports to show sales by product category, in education to represent student grades by subject, etc.

Scenario-Based Question: Suppose You are an education analyst trying to compare the average scores of students across different subjects. Use the following dataset:

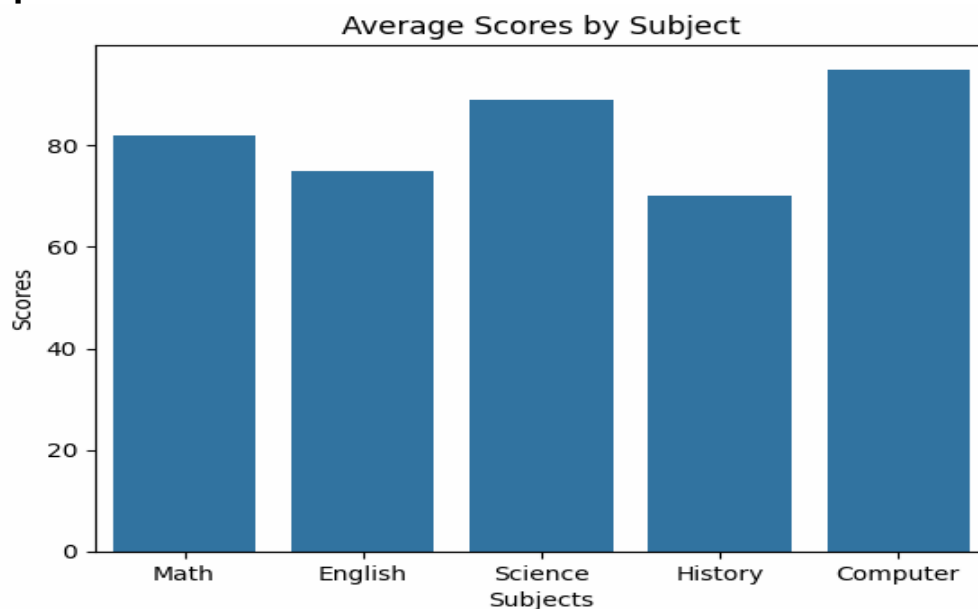
```
Subject:Math,English,Science,History,computer
Average_Score:82,75,89,70,95,
```

Code:

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame from dataset
data = {'Subject': ['Math', 'English', 'Science', 'History', 'Computer'],
        'Average_Score': [82, 75, 89, 70, 95]}
df = pd.DataFrame(data)
sns.barplot(x='Subject', y='Average_Score', data=df)
plt.title('Average Scores by Subject')
plt.xlabel('Subjects')
plt.ylabel('Scores')
plt.show()
```

Output:



2. Line Plot

What is it? Line plots display data points connected by a continuous line and are used to show trends over time.

Where is it used in daily life? Line plots are commonly used in finance to show stock prices, in health to monitor patient vitals, and in weather apps to display temperature changes.

Scenario-Based Question: You're an employee in a multi national company as a sales manager monitoring monthly revenue changes. Use the following dataset:

Month: Jan, Feb, Mar, Apr, May

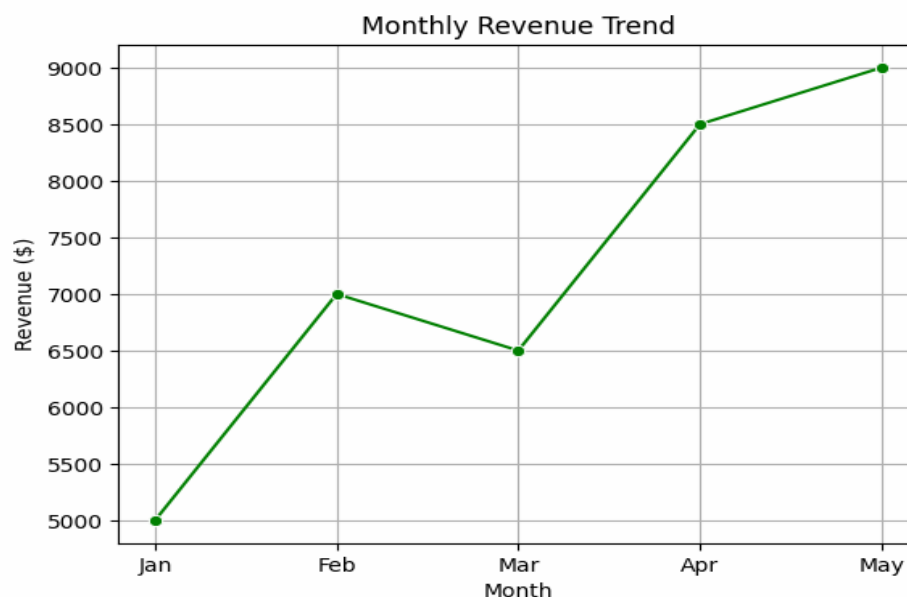
Revenue:5000,7000,6500,8500,9000

Code:

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame from dataset
sales_data = {'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May'],
              'Revenue': [5000, 7000, 6500, 8500, 9000]}
df = pd.DataFrame(sales_data)
# Create line plot
sns.lineplot(x='Month', y='Revenue', data=df, marker='o', color='green')
plt.title('Monthly Revenue Trend')
plt.xlabel('Month')
plt.ylabel('Revenue ($)')
plt.grid(True)
plt.show()
```

Output:



3. Histogram

What is it? Histograms are used to show the distribution of a single numerical variable by grouping data into bins.

Where is it used in daily life? Histograms are used in quality control to check distribution of product dimensions or in exams to analyze the distribution of marks.

Scenario-Based Question: You are a Lab teacher analyzing how students scored in a recent Coding test out of 100. Use the below dataset:

Scores

45,56,67,70,90,85,88,77,50,95

Code:

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame from dataset
data = {'Scores': [45, 56, 67, 70, 90, 85, 88, 77, 50, 95]}
df = pd.DataFrame(data)
# Create histogram
sns.histplot(data=df, x='Scores', bins=5, color='purple')
plt.title('Distribution of coding Test Scores')
plt.xlabel('Scores')
plt.ylabel('Frequency')
plt.show()
```

Output:

